Introduction to Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines document search with a language model. Instead of answering from general model memory alone, the assistant retrieves relevant passages from uploaded documents and includes them in the prompt for answer generation.

The purpose of RAG is to help answers stay grounded in user-provided documents. A typical pipeline extracts PDF text, divides it into chunks, embeds the chunks, retrieves similar text based on a user question, and sends evidence to the language model.

Our example application is called DocuMind AI. It uses Streamlit for its interface. PyMuPDF extracts page text and preserves page numbers for evidence references. The project supports up to five uploaded PDF files and at most 200 combined pages.

How DocuMind Finds Supporting Evidence

DocuMind splits cleaned PDF text into overlapping chunks approximately 1000 characters in length, with about 180 characters of overlap. Overlap preserves nearby context.

The app uses the sentence-transformers/all-MiniLM-L6-v2 embedding model.
Embeddings are numerical vectors representing the meaning of passages and questions.
NumPy cosine similarity compares the question vector against chunk vectors.
By default, the system selects four of the closest matching chunks.

A Gemini model receives the retrieved excerpts and instructions to answer from those excerpts only. The interface labels sources using the form [1], [2], etc.
Users can inspect the original file name, page number, and retrieved excerpt.

The chatbot cannot reliably read image-only scanned PDFs without an OCR step.
Similarity scores reflect vector closeness, not guaranteed answer correctness.